

**DS634 FINAL PROJECT REPOR**

**GHIYAS NIZAMUDDEN SHAIK (gs649)**

**INSTRUCTOR: YASSER ABDUALLAH**

**Table of Contents**

|                                             |          |
|---------------------------------------------|----------|
| <b>Required Software and Packages .....</b> | <b>2</b> |
| <b>Running Instructions.....</b>            | <b>2</b> |
| <b>Code.....</b>                            | <b>3</b> |
| <b>Program Testing .....</b>                | <b>4</b> |
| <b>Results.....</b>                         | <b>4</b> |
| <b>Resources .....</b>                      | <b>4</b> |

## Required Software and Packages

Please install the following packages in the terminal using the following commands.

```
python 3.11.7

numpy 1.26.4 - pip install numpy

pandas 2.2.3 - pip install pandas

scikit-learn 1.6.1 - pip install scikit-learn

tensorflow 2.16.1 - pip install tensorflow
```

## Running Instructions

Please follow the steps to run the program

1. Navigate to the directory where the code is located.
2. Create a folder named **data** within the same directory.
3. Place the provided dataset into the **data** folder.
4. Open the terminal and navigate to the directory containing the code.
5. Run the python file using the command

```
python3 ds634-endsem.py
```

You should see an output like this on the terminal.

```
> python3 ds634-endsem.py
      tpr      tnr      fpr      fnr      recall precision f1-score \
1  0.942737  0.977846  0.022154  0.057263  0.942737  0.970414  0.956375
2  0.939054  0.977078  0.022922  0.060946  0.939054  0.969559  0.954063
3  0.941745  0.976036  0.023964  0.058255  0.941745  0.968354  0.954865
4  0.94386  0.980449  0.019551  0.05614  0.94386  0.974196  0.958788
5  0.937803  0.974044  0.025956  0.062197  0.937803  0.965486  0.951443
6  0.939911  0.976871  0.023129  0.060089  0.939911  0.968914  0.954192
7  0.946691  0.974677  0.025323  0.053309  0.946691  0.965223  0.955867
8  0.943182  0.977126  0.022874  0.056818  0.943182  0.969093  0.955962
9  0.941672  0.977725  0.022275  0.058328  0.941672  0.970071  0.955661
10 0.948402  0.977537  0.022463  0.051598  0.948402  0.968524  0.958357

      accuracy error rate      npv      fdr      bacc      tss      hss
1  0.962564  0.037436  0.956812  0.029586  0.960292  0.920583  0.923602
2  0.960447  0.039553  0.953749  0.030441  0.958066  0.916132  0.919351
3  0.961024  0.038976  0.955589  0.031646  0.958891  0.917781  0.92058
4  0.964392  0.035608  0.95714  0.025804  0.962154  0.924309  0.927457
5  0.958229  0.041771  0.95289  0.034514  0.955923  0.911847  0.914808
6  0.960828  0.039172  0.954946  0.031086  0.958391  0.916782  0.91999
7  0.962753  0.037247  0.960979  0.034777  0.960684  0.921368  0.923652
8  0.962464  0.037536  0.957655  0.030907  0.960154  0.920308  0.923265
9  0.962079  0.037921  0.956261  0.029929  0.959699  0.919397  0.922547
10 0.965255  0.034745  0.962957  0.031476  0.96297  0.925939  0.928555
293/293 _____ 1s 2ms/step
293/293 _____ 1s 2ms/step
```

## Code

This section showcases key parts of the code.

```
# returns confusion matrix values given predicted and ground truth values
```

```
def confusion_matrix(preds, truth):
    tp = np.sum((preds == 1) & (truth == 1))
    fp = np.sum((preds == 1) & (truth == 0))
    tn = np.sum((preds == 0) & (truth == 0))
    fn = np.sum((preds == 0) & (truth == 1))
    return (tp, fp, tn, fn)
```

```
# use k-fold with 10 splits
```

```
kf = KFold(n_splits=10, shuffle=True, random_state=42)
```

### Random Forest

```
rf_metrics = pd.DataFrame([], columns=metric_list)

for i, (train_index, test_index) in enumerate(kf.split(X)):
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    rf = RandomForestClassifier(n_estimators=50)
    rf.fit(X_train, y_train)
    pred_proba = rf.predict_proba(X_test)[:, 1]
    preds = (pred_proba >= 0.5).astype(int)

    tp, fp, tn, fn = confusion_matrix(preds, y_test)
    model_metrics = metrics(tp, fp, tn, fn)
    rf_metrics.loc[i+1,:] = model_metrics

rf_avg = rf_metrics.mean()
```

```
# returns a dictionary of different metrics given the confusion matrix
```

```
def metrics(tp, fp, tn, fn):
    model_metrics = {}
    tpr = tp / (tp + fn)
    model_metrics["tpr"] = tpr
    tnr = tn / (fp + tn)
    model_metrics["tnr"] = tnr
    fpr = fp / (fp + tn)
    model_metrics["fpr"] = fpr
    fnr = fn / (tp + fn)
    model_metrics["fnr"] = fnr

    model_metrics["recall"] = tpr
    prec = tp / (tp + fp)
    model_metrics["precision"] = prec
    f1 = 2 * prec * tpr / (prec + tpr)
    model_metrics["f1-score"] = f1
    acc = (tp + tn) / (tp + fn + fp + tn)
    model_metrics["accuracy"] = acc
    err = (fp + fn) / (tp + fn + fp + tn)
    model_metrics["error rate"] = err
    npv = tn / (tn + fn)
    model_metrics["npv"] = npv
    fdr = fp / (fp + tp)
    model_metrics["fdr"] = fdr

    bacc = (tpr + tnr) / 2
    model_metrics["bacc"] = bacc
    tss = tpr - fpr
    model_metrics["tss"] = tss
    hss = 2 * (tp * tn - fp * fn) / ((tp + fn) * (fn + tn) + (tp + fp) * (fp + tn))
    model_metrics["hss"] = hss

    return model_metrics
```

## Program Testing

Below is the screenshot of the average metrics obtained after testing the 3 models (**Random Forest, Deep Learning: LSTM, ML Algorithms: KNN**) using 10-fold cross validation with a sample dataset.

|            | Random Forest | LSTM     | KNN      |
|------------|---------------|----------|----------|
| tpr        | 0.942506      | 0.92638  | 0.877882 |
| tnr        | 0.976939      | 0.967976 | 0.963009 |
| fpr        | 0.023061      | 0.032024 | 0.036991 |
| fnr        | 0.057494      | 0.07362  | 0.122118 |
| recall     | 0.942506      | 0.92638  | 0.877882 |
| precision  | 0.968984      | 0.956904 | 0.947926 |
| f1-score   | 0.955557      | 0.941354 | 0.911544 |
| accuracy   | 0.962003      | 0.949975 | 0.926048 |
| error rate | 0.037997      | 0.050025 | 0.073952 |
| npv        | 0.956898      | 0.945068 | 0.911345 |
| fdr        | 0.031016      | 0.043096 | 0.052074 |
| bacc       | 0.959722      | 0.947178 | 0.920446 |
| tss        | 0.919445      | 0.894356 | 0.840891 |
| hss        | 0.922381      | 0.897743 | 0.848164 |

## Results

The results show that Random Forest is the best performing model, whereas KNN is the worst. Random Forest performs well in classification issues because of its ensemble nature, which decreases overfitting and improves accuracy. KNN fared badly because it cannot distinguish between crucial and unimportant features, resulting in fewer accurate predictions. Here, the LSTM model did quite well; however, increasing the number of iterations will considerably enhance its performance and may outperform Random Forests.

## Resources

1. GitHub code link: [https://github.com/Ghiyas155/Final\\_project](https://github.com/Ghiyas155/Final_project)
2. Airline Passenger Satisfaction (Kaggle) dataset link:  
<https://www.kaggle.com/datasets/teejmahal20/airline-passenger-satisfaction>